PERSONAL INFORMATION MANAGEMENT (PIM) SYSTEM

Requirements Coverage Report

Document	Requirements Coverage Report
Version	1.4
Date	8 November 2026
Group	Group [number]
Status	Final

Group members

Name	Student ID
[Member 1 name]	[Student ID]
[Member 2 name]	[Student ID]
[Member 3 name]	[Student ID]
[Member 4 name]	[Student ID]

1 Summary

This report states which user stories of Appendix B and which requirements of the Software Requirements Specification (SRS, version 1.4) the implementation covers.

Item	Total	Implemented	Not implemented
User stories (Appendix B)	11	11	0
Functional requirements (FR-1 – FR-43)	43	43	0
Non-functional requirements (NFR-1 – NFR-10)	10	9	1

The evidence column names the automated tests that check each item. Test files are in `tests/unit`, `tests/integration`, and `tests/e2e`; model line coverage is reported separately in `docs/04-unit-tests/coverage.txt`.

2 User stories

Story	Summary	SRS requirements	Status
US1	Create different types of PIR	FR-1 – FR-3, FR-10, FR-11	Implemented
US2	Create Notes (plain text)	FR-4	Implemented
US3	Create Tasks with description and deadline	FR-5	Implemented
US4	Create Events with description, start, and alarms	FR-6 – FR-8, FR-41 – FR-43	Implemented
US5	Create Contacts with name, address, and mobile number	FR-9	Implemented
US6	Modify existing PIRs	FR-12 – FR-15	Implemented
US7	Search by type, text, time, and `&& || !`	FR-16 – FR-23	Implemented
US8	Print one PIR or all PIRs	FR-24 – FR-26	Implemented
US9	Delete a PIR	FR-27, FR-28	Implemented
US10	Store PIRs in a `.pim` file	FR-29 – FR-33	Implemented
US11	Load PIRs from a `.pim` file	FR-34 – FR-37	Implemented

The user-interface requirements FR-38 – FR-40 serve every story.

3 Functional requirements

Id	Requirement (short)	Story	Status	Evidence (tests)
FR-1	Four PIR types; `create [type]`	US1	Implemented	`unit/test_pir.py`; `e2e/test_commands.py`
FR-2	Ids 1, 2, 3, … from the next Id	US1	Implemented	`unit/test_pim.py`

Id	Requirement (short)	Story	Status	Evidence (tests)
FR-3	One prompt per field	US1	Implemented	`integration/test_terminal.py`; `e2e/test_user_flows.py`
FR-4	Note: required text, Display Name	US2	Implemented	`unit/test_pir.py` (`NoteTests`)
FR-5	Task: description, optional deadline	US3	Implemented	`unit/test_pir.py` (`TaskTests`)
FR-6	Event: description, start, 0..n alarms	US4	Implemented	`unit/test_pir.py` (`EventTests`)
FR-7	Relative and absolute alarms	US4	Implemented	`unit/test_alarms.py`; `unit/test_pir.py`
FR-8	Alarm times within years 1–9999	US4	Implemented	`unit/test_alarms.py`; `e2e/test_commands.py`
FR-9	Contact: name, optional address and mobile	US5	Implemented	`unit/test_pir.py` (`ContactTests`)
FR-10	Missing values; messages name the field	US1	Implemented	`unit/test_pir.py`
FR-11	Date-time input in HKT, to the minute	US1	Implemented	`unit/test_pir.py` (`ParseDatetimeTests`)
FR-12	Modify keeps, clears, or replaces each field	US6	Implemented	`integration/test_terminal.py`; `e2e/test_user_flows.py`
FR-13	Id and type never change	US6	Implemented	`unit/test_pir.py`; `unit/test_pim.py`
FR-14	Relative alarms follow the start	US6	Implemented	`unit/test_pir.py`
FR-15	Failed modify changes nothing; no-op stays clean	US6	Implemented	`unit/test_pim.py`
FR-16	Criterion grammar and precedence	US7	Implemented	`unit/test_search.py`
FR-17	`type = T`	US7	Implemented	`unit/test_search.py`
FR-18	`f contains "s"`, case-folded	US7	Implemented	`unit/test_search.py`
FR-19	Unqualified `contains`	US7	Implemented	`unit/test_search.py`
FR-20	`< > =` on instants; missing field is false	US7	Implemented	`unit/test_search.py`
FR-21	`&&`, `||`, `!`, parentheses	US7	Implemented	`unit/test_search.py`
FR-22	Result in Id order; first match selected; `clear`	US7	Implemented	`unit/test_app.py`; `integration/test_app.py`
FR-23	Syntax error keeps the Current Result	US7	Implemented	`unit/test_app.py`; `unit/test_curses_ui.py`
FR-24	`print` shows every field and alarm	US8	Implemented	`integration/test_app.py`; `e2e/test_user_flows.py`
FR-25	`print all` prints the Current Result	US8	Implemented	`integration/test_app.py`; `e2e/test_user_flows.py`
FR-26	Selection errors	US8	Implemented	`integration/test_app.py`
FR-27	Delete after confirmation	US9	Implemented	`e2e/test_user_flows.py`
FR-28	Ids never reused	US9	Implemented	`unit/test_pim.py`; `unit/test_persist.py`
FR-29	`save` / `save as`; `.pim` added; paths	US10	Implemented	`unit/test_persist.py`; `unit/test_app.py`

Id	Requirement (short)	Story	Status	Evidence (tests)
FR-30	Confirm before replacing another file	US10	Implemented	`unit/test_terminal.py`; `integration/test_app.py`
FR-31	UTF-8 JSON `pim/v1` format	US10	Implemented	`unit/test_persist.py`
FR-32	Atomic write; operating-system errors	US10	Implemented	`unit/test_persist.py`; `unit/test_app.py`
FR-33	Empty path and empty file name rejected	US10	Implemented	`unit/test_persist.py`; `unit/test_terminal.py`
FR-34	Load only `.pim`; replaces the collection	US11	Implemented	`unit/test_persist.py`; `unit/test_app.py`
FR-35	Save then load gives the same PIRs	US11	Implemented	`unit/test_persist.py`
FR-36	Invalid file rejected; memory unchanged	US11	Implemented	`unit/test_persist.py`
FR-37	Save / discard / cancel before load or quit	US11	Implemented	`unit/test_terminal.py`; `e2e/test_commands.py`
FR-38	Commands, aliases, and accelerators	all	Implemented	`unit/test_keys.py`; `unit/test_terminal.py`; `e2e/test_commands.py`
FR-39	Folder browser for load and save-as paths	all	Implemented	`unit/test_file_browser.py`; `unit/test_curses_ui.py`; `unit/test_terminal.py`
FR-40	Screen layout; rows are not identities	all	Implemented	`unit/test_layout.py`; `integration/test_terminal.py`; `integration/test_app.py`
FR-41	OVERDUE and SOON Alarm Alerts	US4	Implemented	`unit/test_alarms.py`; `e2e/test_alerts.py`
FR-42	Alerts update within 500 ms	US4	Implemented	inspection of both event loops; `e2e/test_alerts.py`
FR-43	`dismiss`, kept in memory only	US4	Implemented	`unit/test_terminal.py`; `e2e/test_alerts.py`

4 Non-functional requirements

Id	Requirement (short)	Status	Evidence
NFR-1	Python standard library only	Implemented	inspection of the imports in `model/`, `view/`, `controller/`, and `pim.py`
NFR-2	macOS, Python 3.12 or newer	Implemented	the test suite on macOS; the developer manual
NFR-3	`model` package that imports neither `view` nor `controller`	Implemented	inspection of the package layout and imports
NFR-4	A failed command changes nothing, prints no traceback, does not exit, and shows one English message	Implemented	unit, integration, and e2e error tests

Id	Requirement (short)	Status	Evidence
NFR-5	Atomic save; failed load keeps memory; no silent loss of changes	Implemented	`unit/test_persist.py`; `unit/test_terminal.py`; `e2e/test_commands.py`
NFR-6	Time-zone-aware instants, HKT by default, UTC+8 without a time-zone database	Implemented	`unit/test_pir.py` (`ParseDatetimeTests`)
NFR-7	Search, alarm check, save, and load each under 1 s with 10,000 PIRs	Implemented	measured on 24 Sep 2026: 2 ms, 2 ms, 44 ms, 24 ms
NFR-8	Model unit tests with `unittest`, 100 % line coverage of `model/`	Not implemented	One failing assertion in `test_json_too_deep_or_too_long_is_not_a_pim_file`; `coverage.txt` still shows 800 of 800 lines
NFR-9	English text; errors name the field or value	Implemented	`unit/test_pir.py` and the error tests; the user manual, Section 11
NFR-10	No out-of-scope features	Implemented	inspection against SRS Section 2.2

5 Not implemented

All 11 user stories and all 43 functional requirements are implemented. NFR-8 is not fully met: `python3 -m unittest` currently has one failing assertion in `test_json_too_deep_or_too_long_is_not_a_pim_file` (`tests/unit/test_persist.py`), where the expected and actual error messages differ. The unit tests still execute all 800 countable lines of `model/`, but the test suite must pass before NFR-8 can be marked implemented. The features listed as out of scope in SRS Section 2.2 (a graphical interface, recurring events, links between PIRs, operating-system notifications, and third-party terminal libraries) are absent by design.

6 How the evidence was produced

- `python3 -m unittest` runs all three test layers.
- `python3 tests/coverage_report.py` writes `docs/04-unit-tests/coverage.txt`, typeset as `COVERAGE_REPORT.pdf`. It uses the standard-library `trace` module over `tests/unit` and counts the lines on which a statement starts in each file of `model/` except `__init__.py`. The method is stated at the top of `coverage.txt`.